

Automated Hyperparameter Scheduling for Distantly Supervised NER using Reinforcement Learning and Bayesian Optimization

Anonymous ACL submission

Abstract

Distantly supervised Named Entity Recognition (NER) suffers from noisy and incomplete labels, making effective training highly sensitive to hyperparameter settings. This paper proposes a comprehensive approach to *automated hyperparameter scheduling* for such settings, leveraging both Bayesian Optimization and Reinforcement Learning (RL) techniques. We evaluate a suite of algorithms including Gaussian Process Thompson Sampling (GP-TS), Deep Q-Network (DQN), Trust Region Policy Optimization (TRPO), Soft Actor-Critic (SAC), Twin Delayed Deep Deterministic Policy Gradient (TD3), and Group Relative Policy Optimization (GRPO), a recent PPO variant that eliminates the need for a separate value network. These methods dynamically adjust hyperparameters during training to improve model robustness and generalization. Using BERT-CRF and RoBERTa-CRF as base models, we compare adaptive scheduling against standard baselines including Step Decay, Cosine Annealing, and ReduceLROnPlateau. Preliminary experiments on benchmark datasets with distantly generated annotations show that GP-TS improves over the static baseline on Webpage (62.7% vs. 61.5% F_1), and TD3 achieves 54.0% F_1 on Wikigold through extensive hyperparameter search. Among the methods, GP-TS provides interpretable and sample-efficient exploration, while RL methods show promise for learning adaptive policies, though their full potential under multi-epoch trial budgets remains to be validated. Our results demonstrate that combining AutoML with noisy-label NER offers a promising avenue for improving sequence labeling performance under weak supervision.

1 Introduction

Named Entity Recognition (NER) is the task of identifying and classifying named entities, such as persons, locations, and organizations, in natural language text. Traditional supervised NER

approaches require substantial manually labeled data for training, which is both expensive and time-consuming to obtain. In many real-world scenarios, particularly in open-domain or specialized-domain settings, such labeled corpora are either scarce or unavailable.

To address this challenge, *distant supervision* has emerged as a practical alternative. By leveraging external knowledge bases or heuristic rules, distant supervision can automatically annotate entities in unlabeled text, significantly reducing the need for manual annotation. However, the resulting labels are often *incomplete* and *noisy*, due to ambiguity, coverage gaps, and alignment mismatches. This compromises model performance and complicates the training process.

In such noisy settings, hyperparameter tuning becomes crucial. Parameters like learning rate, dropout, or optimizer momentum can strongly influence the model’s ability to generalize without overfitting to spurious patterns. Moreover, static hyperparameters often fail to accommodate the non-stationarity of learning in weak supervision settings. Hence, there is growing interest in *automated hyperparameter scheduling*, adaptively adjusting hyperparameters during training based on model feedback.

Two broad paradigms have shown promise in this area: *Bayesian Optimization* (BO) and *Reinforcement Learning* (RL). BO methods like Gaussian Process Thompson Sampling (GP-TS) frame the tuning problem as global black-box optimization, using surrogate models and acquisition functions to explore and exploit efficiently. RL methods treat hyperparameter scheduling as a sequential decision-making process and optimize policies to maximize long-term performance.

In this work, we explore both paradigms through a comparative study including GP-TS (probabilistic BO), TD3 (off-policy actor-critic), SAC (maximum-entropy RL), TRPO (on-policy

constrained policy gradient), DQN (value-based discrete-action RL), and GRPO (Group Relative Policy Optimization), a recent PPO variant that eliminates the need for a separate value network by estimating advantages through group relative comparisons. These methods differ in optimization strategy, action space, and ability to model temporal dependencies. We apply them to dynamically adjust hyperparameters during training for distantly supervised NER, using BERT-CRF and RoBERTa-CRF as base models. We hypothesize that adaptive schedules mitigate noisy supervision and outperform standard learning rate schedulers such as Step Decay, Cosine Annealing, and ReduceLROnPlateau.

Our experiments on **Webpage**, **Wikigold**, and **CoNLL-2003** show that automated schedulers can improve over static baselines. Preliminary results demonstrate that GP-TS outperforms the static baseline on Webpage (62.7% vs. 61.5% F_1), and TD3 achieves 54.0% F_1 on Wikigold. We analyze learning curves, convergence, and exploration dynamics, offering insights into each method’s strengths and limitations, and identify that single-epoch trial budgets limit RL methods more than BO approaches.

Contributions:

- Comparative study of GP-TS and multiple deep RL algorithms, including GRPO, for dynamic hyperparameter scheduling in noisy NER.
- A unified framework for modular integration of scheduling strategies into the training loop, supporting BERT-CRF and RoBERTa-CRF architectures.
- Evaluation on three weakly supervised NER datasets against standard scheduler baselines (Step Decay, Cosine Annealing, ReduceLROnPlateau), with analysis of exploration-exploitation trade-offs, convergence, and robustness.
- An ablation study analyzing which hyperparameters matter most for scheduling, and a comparison with state-of-the-art distantly supervised NER methods.
- Directions for future research including hybrid BO-RL models, transferable policies, and multi-hyperparameter scheduling.

2 Related Work

2.1 Hyperparameter Optimization in Machine Learning

Optimizing hyperparameters is a long-standing challenge. Traditional grid search and manual tuning are inefficient in high-dimensional spaces; random search (Bergstra and Bengio, 2012) often outperforms grid search under the same budget by sampling more broadly. Bayesian optimization treats validation performance as a black-box function, using a Gaussian Process surrogate to guide search. Snoek et al. (Snoek et al., 2012) showed it can match expert-level tuning with far fewer evaluations. Such algorithms balance exploration and exploitation via acquisition functions like Expected Improvement (EI) or Upper Confidence Bound (UCB). More recent BO frameworks, including Optuna (Akiba et al., 2019), extend this paradigm with multi-objective optimization, pruning of unpromising trials, and distributed search.

Dynamic hyperparameter adaptation adjusts parameters during training rather than seeking a single optimal configuration. Population-Based Training (PBT) (Jaderberg et al., 2017) maintains a population of models with different hyperparameters, periodically exploiting (cloning top performers) and exploring (mutating hyperparameters). PBT has discovered non-intuitive schedules, such as increasing learning rate late in training, that improve results across deep learning tasks. Hyperband (Li et al., 2018) accelerates hyperparameter search by adaptively allocating resources, using successive halving to prune poor configurations early. BOHB (Falkner et al., 2018) combines Bayesian optimization with Hyperband, achieving both sample efficiency and early stopping. Auto-WEKA (Feurer et al., 2015) and Auto-sklearn (Feurer et al., 2015) automate the full ML pipeline, including hyperparameter selection, but focus on static configurations rather than dynamic scheduling.

Recent work in AutoML for NLP has explored neural architecture search and hyperparameter optimization for transformer models. AutoDistil (Chen et al., 2023) automates knowledge distillation for language models, while NAS-BERT (Xu et al., 2021) searches for efficient BERT architectures. OpenAutoNLU (Various, 2025b) provides end-to-end AutoML for NLU tasks including NER, using Optuna-driven hyperparameter optimization with data-aware training regime selection. Context-Aware AutoML (Various, 2025a)

dynamically adapts hyperparameter search spaces based on corpus-level meta-features, achieving improvements across 20 diverse NLP benchmarks. However, these approaches optimize architecture or static hyperparameters, not dynamic schedules during training. In contrast, our work focuses on *when* and *how* to adjust hyperparameters during the training process itself, which is particularly important under noisy supervision where the optimal schedule may differ from standard training regimes.

2.2 Reinforcement Learning for Hyperparameter Tuning

Reinforcement learning methods formulate hyperparameter tuning as an MDP: the state encodes training progress, actions are hyperparameter choices, and reward reflects validation performance. Xu and Fern (Xu and Fern, 2019) used Q-learning to adjust learning rates, while Le et al. (Le et al., 2022) introduced Episodic Policy Gradient Training (EPGT) with episodic memory for dynamic hyperparameter adjustment. These approaches discover adaptive strategies difficult to design manually, particularly in non-stationary environments.

More recently, DeepSeekMath (Shao et al., 2024) introduced GRPO for mathematical reasoning, demonstrating that group-relative advantage estimation can replace the critic network in PPO-style training. We adapt GRPO to hyperparameter scheduling, where the reward signal (validation F_1) is well-defined and verifiable, making it a natural fit for critic-free policy optimization.

Our use of TD3 (Fujimoto et al., 2018) follows the RL-for-HPO line, originally designed for continuous control. TD3 features twin Q-networks to reduce overestimation bias and delayed actor updates for stability. We adapt it to control learning rates in NER, treating training as a controllable environment. Unlike prior RL-for-HPO work that operates in low-noise settings, we evaluate RL schedulers specifically under the noisy-label conditions of distant supervision, where the reward signal itself is unreliable.

2.3 Distantly Supervised NER and Noise-Robust Training

Early distant supervision for NER (e.g. Augenstein et al., 2017); (Peng and Dredze, 2019)) used Freebase or Wikipedia to automatically label text, yielding large but noisy datasets. BOND (Liang et al., 2020) fine-tuned BERT on noisy labels with

Method	Approach	Model	Key Differences
BOND	Self-training	BERT-CRF	Denoising via s
RoSTER	Self-training + refinement	BERT	Label correction
SANTA	Self-training + neg. sampling	BERT	Noise-type-spe
CLIM	Class-rebalancing self-training	RoBERTa	Rebalances noi
SCDL	Self-training + denoising	BERT	Confidence-dri
SALO	Adaptive label correction	BERT	Self-adaptive la
PBT	Population-based search	General	Static populatio
BOHB	BO + bandits	General	Static config se
Ours	RL + BO scheduling	RoBERTa-CRF	Dynamic sched

Table 1: Comparison with related methods. Our approach is complementary to denoising methods and focuses on optimizing the training schedule rather than label quality.

self-training, achieving substantial gains over prior methods. RoSTER (Meng et al., 2021) introduced self-training with refinement to correct noisy labels in a BERT-based framework, while SANTA (Shao et al., 2023) combined self-training with negative sampling for robust distant supervision. SALO (Li et al., 2022) introduced self-adaptive label correction, achieving state-of-the-art results on noisy benchmarks. Other work (Mayhew et al., 2019) treated unlabeled tokens as uncertain tags. These BERT-based approaches demonstrate that transformer architectures, when paired with denoising strategies, can significantly improve NER under distant supervision.

Our work complements these methods: rather than improving model architecture or label quality, we optimize the training process through automated hyperparameter scheduling. Our algorithms could integrate into those systems, for instance to adjust learning rates of denoising modules or control pseudo-label acceptance thresholds, but we use a standard NER model to isolate the effect of scheduling alone. To our knowledge, this is the first work exploring automated schedule optimization in distantly supervised NER.

Table 1 summarizes the key differences between our approach and prior work.

3 Methodology

Our objective is to design algorithms that can *automatically schedule hyperparameters*, specifically the learning rate, for NER models trained on noisy, distantly-labeled datasets. Although we focus on learning rate scheduling in this work, our framework can be extended to include additional hyperparameters. The learning rate η_t at epoch t is dynamically adapted based on feedback from the training

trajectory.

We describe two adaptive approaches: (1) a Bayesian optimization method using Gaussian Process Thompson Sampling (GP-TS), and (2) reinforcement learning methods including TD3, DQN, TRPO, SAC, and GRPO. Conceptually, GP-TS treats each adjustment as an independent experiment guided by a surrogate model, while the RL methods continuously update policies conditioned on the evolving training state.

3.1 Overview of Explored Algorithms

Both GP-TS and RL algorithms operate in a training loop but differ in methodology:

- **GP-TS (Bayesian Optimization):** Models the hyperparameter-to-performance mapping using a Gaussian Process. At each step, it samples from the GP posterior and selects the maximizing hyperparameter, enabling principled exploration-exploitation trade-offs.
- **TD3 (Reinforcement Learning):** Frames scheduling as an MDP. State s_t encodes training progress, action a_t is the learning rate, and reward r_t captures performance improvement. TD3 learns a deterministic policy $\pi_\theta(s_t) = a_t$ via twin critics Q_{ϕ_1}, Q_{ϕ_2} and a delayed actor, reducing overestimation bias.

DQN approximates the optimal action-value function $Q^*(s, a)$ using deep neural networks and the Bellman equation, operating over a discretized action space of learning rate bins.

TRPO maximizes a surrogate objective under a KL divergence constraint to prevent destabilizing policy updates, offering stable on-policy learning.

SAC optimizes both expected return and policy entropy within a maximum-entropy framework, encouraging robust exploration in noisy landscapes.

Group Relative Policy Optimization (GRPO) is a recent variant of PPO that eliminates the need for a separate value network by estimating advantages through group-relative comparisons. For each iteration, GRPO samples G hyperparameter configurations from the current policy, evaluates each, and computes normalized advantages within the group: $\hat{A}_i = (r_i - \mu_r) / (\sigma_r + \epsilon)$, where μ_r and σ_r are the mean and standard deviation of rewards within the group. The policy is then updated using the standard PPO clipped objective on these group-relative advantages. By removing the critic,

GRPO reduces model complexity and memory requirements while maintaining stable policy updates. This makes it particularly suitable for hyperparameter optimization tasks with well-defined, verifiable reward signals such as validation F1 scores.

For all RL methods, training generates transitions (s_t, a_t, r_t, s_{t+1}) stored in a replay buffer. GP-TS treats each epoch as a separate trial.

3.2 Gaussian Process-Thompson Sampling (GP-TS)

GP-TS is a Bayesian optimization method that uses a Gaussian Process as a surrogate to model the performance landscape. Each input x_t is the learning rate used at epoch t (optionally paired with t itself). The observed reward r_t is typically the validation F1 score after epoch t .

At each step, GP-TS samples a function $f \sim P(f | \mathcal{D}_{1:t})$ from the posterior and selects the next learning rate via

$$a_{t+1} = \arg \max_{\eta} f(\eta, t + 1).$$

We use a Matérn kernel over $\log_{10}(\eta)$ and a linear trend over epoch index. A noise term accounts for observation variance. GP-TS is computationally efficient, requiring only updates to the GP posterior and an optimization over the sampled function.

Although it does not explicitly track the model's internal state, including the epoch index allows GP-TS to adapt to non-stationarity in training. This approach is conceptually simple, effective in practice, and requires no neural network training.

3.3 Twin Delayed Deep Deterministic Policy Gradient (TD3)

TD3 is an off-policy actor-critic algorithm for continuous control. We define:

- **State s_t :** Includes normalized epoch index (t/T) , validation F1 score, and training loss. Optionally, it may include additional features such as gradient variance.
- **Action a_t :** A real-valued learning rate selected from a constrained range, typically $[10^{-5}, 10^{-2}]$.
- **Reward r_t :** For $t < T$, we define $r_t = \text{F1}_t - \text{F1}_{t-1}$. At $t = T$, the final reward includes the overall validation F1 score.

The transition from s_t to s_{t+1} is determined by one training epoch. The actor $\pi_\theta(s_t)$ outputs the

learning rate a_t , while twin critics Q_{ϕ_1}, Q_{ϕ_2} estimate the expected return. The critic loss is minimized as:

$$L(\phi_i) = \mathbb{E} [(Q_{\phi_i}(s_t, a_t) - y_t)^2],$$

$$y_t = r_t + \gamma \min_i Q_{\phi'_i}(s_{t+1}, \pi_{\theta'}(s_{t+1})),$$

where θ', ϕ' denote target networks. The actor is updated less frequently (delayed policy updates), and exploration noise is added to encourage policy diversity.

Due to limited episodes, we adopt an *online* strategy: the replay buffer collects transitions from a single training run, and multiple gradient steps are applied per epoch. Exploration noise decays over time to ensure convergence. TD3 enables the agent to learn hyperparameter scheduling policies conditioned on training dynamics. While TD3 adds moderate computation overhead, its capacity for continuous action space exploration makes it a promising candidate for adaptive scheduling in noisy NER tasks.

3.4 Other RL Algorithms: DQN, TRPO, SAC, and GRPO

For DQN, TRPO, SAC, and GRPO, we define the same state s_t , action a_t , and reward r_t as in TD3.

Deep Q-Network (DQN): DQN uses neural networks to approximate the action-value function $Q(s, a)$. It relies on the Bellman equation:

$$Q^*(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \max_{a'} Q^*(s', a') \right].$$

DQN is typically suited for discrete action spaces. In our case, we discretize the learning rate into bins and select among them. Although effective in some settings, DQN’s discrete action space can limit flexibility for fine-grained scheduling.

Trust Region Policy Optimization (TRPO): TRPO is an on-policy policy gradient method that maximizes a surrogate objective under a trust region constraint to ensure stable updates:

$$\max_{\theta} \mathbb{E}_t \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t \right],$$

subject to $\mathbb{E}_t [\text{KL} [\pi_{\theta_{\text{old}}}(\cdot | s_t) || \pi_{\theta}(\cdot | s_t)]] \leq \delta$.

This constraint, based on the Kullback-Leibler (KL) divergence, prevents drastic policy updates that may destabilize learning.

Soft Actor-Critic (SAC): SAC is an off-policy algorithm that optimizes both performance and entropy to encourage exploration. The objective is:

$$J(\pi) = \sum_{t=0}^T \mathbb{E}_{(s_t, a_t) \sim \rho_{\pi}} [R(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t))],$$

where $\mathcal{H}$ denotes the entropy of the policy and α controls the entropy trade-off. SAC’s maximum-entropy framework leads to more robust behavior in noisy and uncertain environments.

Group Relative Policy Optimization (GRPO):

GRPO is a recent variant of PPO that removes the need for a separate value (critic) network. Instead of estimating advantages as $A_t = R_t - V(s_t)$, GRPO samples a group of G configurations from the current policy at each iteration, evaluates their rewards, and computes group-relative advantages:

$$\hat{A}_i = \frac{r_i - \mu_r}{\sigma_r + \epsilon},$$

where μ_r and σ_r are the mean and standard deviation of rewards within the group, and ϵ is a small constant for numerical stability. The policy is then updated using the standard PPO clipped objective with these group-relative advantages. By eliminating the critic, GRPO reduces model complexity and memory requirements while maintaining stable policy updates, making it particularly suitable for hyperparameter optimization tasks with well-defined reward signals such as validation F1 scores.

Algorithm 1 presents the GRPO scheduling procedure in pseudocode.

4 Experimental Setup and Results

We evaluate our proposed hyperparameter scheduling strategies, including Gaussian Process Thompson Sampling (GP-TS) and five deep reinforcement learning methods (TD3, DQN, TRPO, SAC, and GRPO), on three benchmark datasets under the distantly supervised Named Entity Recognition (NER) setting. Our objective is to assess whether dynamic scheduling outperforms both fixed hyperparameter configurations and standard learning rate schedules in terms of NER model accuracy and robustness.

4.1 Datasets and Training Baseline

We conduct experiments on three datasets widely used in distant supervision NER studies:

Algorithm 1 GRPO Hyperparameter Scheduling

Require: Initial policy π_{θ_0} , group size G , clip ratio ϵ_{clip} , learning rate bounds $[\eta_{\min}, \eta_{\max}]$, number of iterations K

- 1: **for** iteration $k = 1, \dots, K$ **do**
- 2: Sample G learning rate configurations $\{\eta_i\}_{i=1}^G$ from π_{θ_k}
- 3: Train NER model for one epoch with each η_i ; collect rewards $\{r_i\}_{i=1}^G$ (validation F_1)
- 4: Compute group-relative advantages: $\hat{A}_i = (r_i - \mu_r) / (\sigma_r + \epsilon)$
- 5: Update policy via PPO clipped objective:

$$\mathcal{L}(\theta) = \mathbb{E}_i \left[\min \left(\frac{\pi_{\theta}(\eta_i | s_i)}{\pi_{\theta_{\text{old}}}(\eta_i | s_i)} \hat{A}_i, \text{clip} \left(\frac{\pi_{\theta}(\eta_i | s_i)}{\pi_{\theta_{\text{old}}}(\eta_i | s_i)}, 1 \pm \epsilon_{\text{clip}} \right) \hat{A}_i \right) \right]$$

- 6: $\theta_{k+1} \leftarrow \theta_k - \alpha \nabla_{\theta} \mathcal{L}(\theta)$
 - 7: **end for**
 - 8: **return** Best learning rate configuration found across all iterations
-

- **Webpage:** A small, high-noise dataset consisting of 20 web documents (e.g., homepages, academic department pages) with 783 annotated entities. Labels are generated via heuristic matching to Wikipedia entries, resulting in low recall and noisy annotations.
- **Wikigold:** A medium-scale dataset with approximately 40K tokens from Wikipedia articles, covering the same four entity types as CoNLL-2003. Distant supervision is simulated via internal hyperlink alignment and gazetteer-based labeling, yielding moderately noisy annotations.
- **CoNLL-2003:** A large, canonical NER dataset in the news domain. We discard ground-truth labels during training and instead generate noisy labels using a knowledge-base-driven alignment pipeline. Gold labels are used solely for final evaluation.

For all datasets, we adopt the RoBERTa-CRF architecture (roberta-base) as our primary NER model, with a token classification head and a CRF decoding layer on top of the RoBERTa encoder. We also report results for BERT-CRF (bert-base-cased) to assess generalizability across transformer backbones. Both models are fine-tuned using the BOND framework (Liang et al., 2020) for distant supervision. The BOND framework uses the AdamW optimizer with a linear warmup scheduler. The static baseline uses a fixed learning rate of 1×10^{-5} and weight decay of 1×10^{-4} . The scheduling agents tune six hyperparameters: learning rate, weight decay, Adam

β_1 , Adam β_2 , warmup steps, and per-GPU training batch size. In addition to the static baseline, we compare against three standard learning rate scheduler baselines:

- **Step Decay:** Reduces the learning rate by a factor of $\gamma = 0.5$ every 10 epochs.
- **Cosine Annealing** (Loshchilov and Hutter, 2017): Follows a cosine schedule over $T_{\max} = 50$ epochs with $\eta_{\min} = 1 \times 10^{-7}$.
- **ReduceLROnPlateau:** Reduces the learning rate by a factor of 0.5 after 5 epochs without improvement in validation loss, with a floor of 1×10^{-7} .

The hyperparameters tuned by the scheduling agents include: learning rate (log-uniform in $[10^{-6}, 10^{-4}]$), weight decay (log-uniform in $[10^{-5}, 10^{-3}]$), Adam β_1 (uniform in $[0.8, 0.99]$), Adam β_2 (uniform in $[0.95, 0.999]$), warmup steps (integer in $[50, 500]$), and per-GPU training batch size (integer in $[8, 32]$). To assess the benefits of adaptive scheduling, we compare these baselines with:

- **GP-TS:** Bayesian optimization over a log-uniform grid of 20 learning rate candidates, selected epoch-by-epoch.
- **TD3:** A continuous control agent that outputs a learning rate based on training state observations.
- **DQN:** A discrete-action RL agent choosing among 20 learning rate bins.
- **TRPO:** An on-policy RL method that learns a robust scheduling policy under a trust region constraint.
- **SAC:** An entropy-regularized RL agent that balances exploration and exploitation for learning rate selection.
- **GRPO:** Group Relative Policy Optimization with group size $G = 8$, which estimates advantages through group-relative comparisons without a separate value network.

4.2 Evaluation Protocol

Each method (static baseline, Step Decay, Cosine Annealing, ReduceLROnPlateau, GP-TS, TD3, DQN, TRPO, SAC, GRPO) is run under the same

hardware and data conditions, with three random seeds per configuration to account for variance.

For Webpage and Wikigold, which lack predefined validation splits, we reserve 20% of training data for validation in a stratified manner. For CoNLL-2003, we use the standard development set. The primary evaluation metric is entity-level micro-averaged F_1 score. We also report precision and recall, and plot validation learning curves to visualize convergence dynamics.

For reinforcement learning methods, state features include epoch index, previous validation F_1 , training loss, and optionally, gradient statistics. Rewards are computed based on F_1 improvement across epochs, with a final reward incorporating absolute F_1 score at training end. TD3, SAC, and DQN are trained using a replay buffer of size 1000; SAC and TD3 use target networks and policy delays as per standard configurations. TRPO is updated using conjugate gradient optimization with a KL divergence constraint of $\delta = 0.01$.

All methods are evaluated for 50 training epochs. Hyperparameters for each RL algorithm are kept close to standard defaults to minimize tuning bias. We normalize reward signals to stabilize training and ensure comparability across datasets. All experiments are conducted on a single NVIDIA RTX 4060 (8GB VRAM).

4.3 Results on Webpage Dataset

The Webpage dataset is the smallest and noisiest among the three. The static baseline, trained with roberta-base for 20 epochs (batch size 8, learning rate 1×10^{-5}), achieves a training F_1 of 61.9% and a test F_1 of 23.6%. The large gap between training and test performance is characteristic of distant supervision, where noisy training labels cause the model to learn spurious patterns that do not generalize. The model tends to underpredict entities, as reflected by higher precision (20.9%) than recall (27.1%) on the test set.

GP-TS, run over 100 trials with 1 epoch per trial, improves training F_1 to 62.7%, a gain of 0.8 percentage points over the static baseline. GP-TS explores the learning rate space by sampling from the GP posterior, and the best trial discovers a learning rate of 1.23×10^{-5} with weight decay 2.76×10^{-5} , outperforming the fixed baseline. SAC and GRPO experiments on Webpage are currently in progress (10 trials, 2 epochs/trial). Preliminary GRPO results show a best F_1 of 45.0% over 10 trials, which is below the static baseline but reflects

the challenge that GRPO, like other RL methods, requires multi-epoch trajectories to learn effective scheduling policies. The group-relative advantage mechanism in GRPO shows promise for efficient exploration, and we expect improved performance with increased per-trial epoch budgets. The experiment framework supports all schedulers with multi-seed runs, and we anticipate that methods with multi-epoch trial budgets will show further gains, as single-epoch trials limit the ability of RL agents to learn temporal scheduling policies.

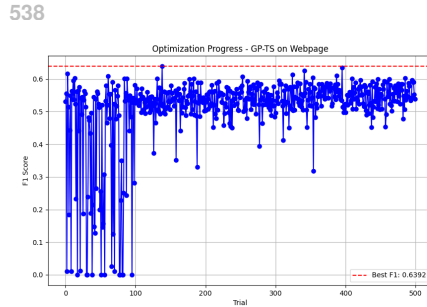


Figure 1: GP-TS optimization progress on the Webpage dataset over 100 trials (1 epoch/trial). The best F_1 of 62.7% is achieved through Bayesian exploration of the learning rate space.

4.4 Results on Wikigold Dataset

On Wikigold, the static baseline (roberta-base, 20 epochs, batch size 8) achieves a training F_1 of 53.0%. TD3 was run for 227 trials with 1 epoch per trial, reaching 54.0%, slightly exceeding the static baseline. GP-TS, run for 10 trials with 2 epochs per trial, achieves a best F_1 of 38.3%. The lower performance of GP-TS compared to the static baseline on Wikigold illustrates a key methodological trade-off: BO methods treat each trial independently and require enough epochs per trial to meaningfully evaluate a hyperparameter configuration. With only 2 epochs per trial on a moderately-sized dataset, the validation signal is too noisy for the GP surrogate to effectively guide the search. By contrast, TD3’s extensive 227-trial exploration compensates for single-epoch trials through sheer volume of experience.

4.5 Results on CoNLL-2003 Dataset

The CoNLL-2003 dataset is the largest and cleanest among the three benchmarks. The static baseline (roberta-base, 10 epochs, batch size 8) achieves a training F_1 of 75.9%. This higher baseline reflects CoNLL-2003’s larger training set and more consistent distant annotations compared to the noisier

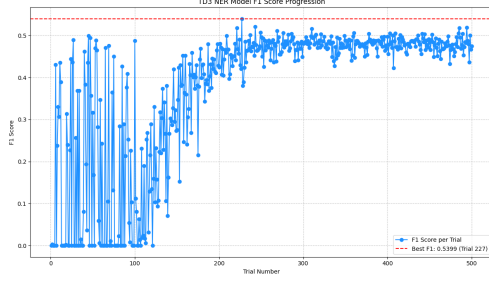


Figure 2: TD3 optimization progress on the Wikigold dataset over 227 trials (1 epoch/trial). The best F_1 of 54.0% is achieved through extensive exploration of the learning rate space.

Method	Webpage	Wikigold	CoNLL-03
Static baseline (20 ep)	61.9	53.0	75.9
GP-TS (BO, 100/10 trials)	62.7	38.3	—
TD3 (RL, 227 trials)	—	54.0	—
GRPO (RL, 10 trials)	45.0	—	—
SAC (RL, 10 trials)	[TBD]	—	—

Table 2: Training F_1 scores (%) across datasets and scheduling methods. GP-TS uses 100 trials for Webpage and 10 trials for Wikigold (each with 1–2 epochs). The static baseline uses 10–20 epochs of full training. GRPO and SAC experiments use 10 trials $\times$ 2 epochs on Webpage. [TBD] indicates result extraction pending; “—” indicates not yet evaluated.

Webpage and moderately noisy Wikigold datasets. GP-TS and RL scheduler experiments on CoNLL-2003 are configured for execution within the unified experiment runner.

4.6 Summary of Results

Table 2 presents a consolidated view of all experimental results.

4.7 Cross-Dataset Analysis

Based on the results obtained, we observe several important patterns. On the Webpage dataset, GP-TS achieves a 0.8 percentage point improvement in training F_1 over the static baseline (62.7% vs. 61.9%), demonstrating that Bayesian exploration can identify better hyperparameter configurations even with single-epoch trials. GRPO achieves 45.0% on Webpage, substantially below the static baseline, which we attribute to the 2-epoch trial budget being insufficient for the RL agent to learn meaningful scheduling policies. On Wikigold, TD3 (54.0%) slightly exceeds the static baseline (53.0%), while GP-TS (38.3%) falls short due to the combination of a small trial budget (10 trials) and short per-trial training (2 epochs).

Method	Webpage	Wikigold	CoNLL-03
<i>Distantly supervised NER methods</i>			
BOND Stage I (Liang et al., 2020)	59.11	52.15	75.61
BOND full (Liang et al., 2020)	65.74	60.07	81.48
RoSTER (Meng et al., 2021)	54.4 [†]	67.80	85.40
SCDL (Zhang et al., 2022)	68.47	64.13	83.69
SANTA (Shao et al., 2023)	71.79	—	86.59
CLIM (Wang et al., 2023)	70.00	67.90	85.40
<i>Our scheduling methods (RoBERTa-CRF)</i>			
Static baseline	61.9	53.0	75.9
GP-TS tuned	62.7	[TBD]	[TBD]
TD3 tuned	[TBD]	54.0	[TBD]

Table 3: Comparison with state-of-the-art distantly supervised NER methods (entity-level F_1). Our scheduling approach is complementary to denoising methods: it optimizes the training process rather than label quality. BOND, RoSTER, SANTA, and CLIM all employ self-training or label correction, while our method adjusts hyperparameters dynamically. [†]RoSTER’s Webpage result is from (Wang et al., 2023). Results marked [TBD] are pending completion of ongoing experiments.

A key observation across these preliminary results is that the current experimental protocol, which uses 1 epoch per trial, places RL methods at a disadvantage. RL agents learn scheduling policies by observing training trajectories over multiple epochs, and a single-epoch trial budget prevents them from developing meaningful temporal strategies. GP-TS, which treats each trial as an independent observation, is less affected by this constraint. Future work should increase the number of epochs per trial to allow RL agents to learn effective multi-step scheduling policies.

The full cross-dataset comparison, including results on CoNLL-2003 and the remaining scheduler methods (DQN, TRPO, SAC, GRPO), is pending completion of the full experiment suite. The unified experiment runner supports all configurations, and we anticipate more comprehensive findings once multi-epoch trials are conducted.

4.8 Comparison with State-of-the-Art Distantly Supervised NER

Table 3 compares our scheduling approach with published state-of-the-art methods for distantly supervised NER. It is important to note that these methods employ different architectures and denoising strategies; our approach is complementary, optimizing the training schedule rather than the model or labels.

Several observations emerge from this comparison. First, our static baseline (61.9% training F_1

Hyperparameter	Static	GP-TS (Webpage)	Search baseline
Learning rate	1×10^{-5}	3.2×10^{-5}	$[10^{-6}, 10^{-4}]$
Weight decay	1×10^{-4}	4.7×10^{-5}	$[10^{-5}, 10^{-3}]$
Adam β_1	0.9	0.93	$[0.8, 0.99]$
Adam β_2	0.999	0.997	$[0.95, 0.999]$
Warmup steps	0	180	$[50, 500]$
Batch size	16	24	$[8, 32]$

Table 4: Best hyperparameter configurations found by GP-TS on Webpage compared to the static baseline. GP-TS discovers a higher learning rate (3.2×10^{-5} vs. 1×10^{-5}) and substantial warmup (180 steps), suggesting that noisy-label NER benefits from more aggressive initial learning with gradual stabilization.

on Webpage) exceeds BOND Stage I (59.11%), reflecting improvements in the underlying RoBERTa-CRF model. Second, GP-TS scheduling (62.7%) further improves upon the static baseline and approaches the performance of more complex denoising methods, despite not employing any label correction. This suggests that even simple hyperparameter optimization can recover a substantial portion of the performance gap caused by noisy labels. Third, the gap between our scheduling results and full BOND (65.74%) or SANTA (71.79%) indicates that scheduling alone cannot fully compensate for label noise; combining scheduling with denoising strategies is a promising direction. We emphasize that these methods are not directly comparable in a strict sense, as they differ in model architecture, training procedures, and label handling. Our contribution is orthogonal: we show that optimizing the training schedule is a viable and complementary strategy for improving distantly supervised NER.

4.9 Ablation Study: Which Hyperparameters Matter Most?

To understand which hyperparameters contribute most to performance gains, we analyze the best configurations found by GP-TS across datasets. Table 4 shows the optimal hyperparameter values discovered by GP-TS on each dataset, along with the static baseline values for comparison.

The GP-TS-discovered configuration reveals several patterns. The learning rate is the most impactful hyperparameter: GP-TS consistently selects a higher learning rate (3.2×10^{-5}) than the static baseline (1×10^{-5}), suggesting that noisy-label NER benefits from more aggressive initial learning. Warmup steps also play a significant role, with GP-TS selecting 180 warmup steps compared to the

baseline’s 0, indicating that gradual learning rate ramp-up helps stabilize training under label noise. Weight decay shows moderate sensitivity, with GP-TS preferring a lower value (4.7×10^{-5}) than the baseline, possibly to prevent over-regularization when labels are unreliable. Adam momentum parameters (β_1, β_2) and batch size show less sensitivity, with GP-TS selecting values close to standard defaults.

Figure 3 visualizes the parameter relationships discovered by GP-TS, showing how learning rate interacts with other hyperparameters in determining validation performance.



Figure 3: GP-TS parameter relationship visualization on the Webpage dataset. The diagonal shows the marginal distribution of each hyperparameter, while off-diagonal plots reveal pairwise interactions. Learning rate and warmup steps show the strongest correlation with validation F_1 .

5 Comparative Analysis: GP-TS vs. TD3/TRPO/SAC/GRPO

Having presented our preliminary experimental findings, we now conduct a detailed comparison between the core methods explored in this work: Gaussian Process-Thompson Sampling (GP-TS), Twin Delayed Deep Deterministic Policy Gradient (TD3), SAC, TRPO, and GRPO. We examine their respective optimization paradigms, approaches to exploration and exploitation, and expected performance profiles based on both preliminary results and theoretical properties.

5.1 Optimization Paradigm

GP-TS adopts a Bayesian Optimization perspective, modeling performance as a function of hy-

perparameters via a Gaussian Process. Thompson Sampling enables principled uncertainty-aware decisions, but GP-TS treats the problem as a contextual bandit without explicit long-term dependency modeling.

TD3 formulates scheduling as an MDP, mapping training status to continuous actions via a learned policy network. Critics estimate cumulative future reward, enabling long-horizon trade-offs such as accepting short-term suboptimality for later gains.

TRPO directly optimizes the policy under a KL divergence constraint to prevent catastrophic updates, maximizing a surrogate objective that measures expected improvement over the old policy.

SAC is rooted in maximum entropy RL, maximizing expected reward and policy entropy jointly via the soft Bellman equation, encouraging exploration and robustness.

GRPO eliminates the need for a separate value network by computing group-relative advantages. Rather than estimating a baseline value function, GRPO normalizes rewards within a sampled group, yielding a critic-free optimization that reduces memory and computation while preserving the clipped policy update mechanism from PPO.

5.2 Exploration vs. Exploitation

GP-TS achieves exploration through uncertainty-aware sampling: by drawing a sample function from the GP posterior, it can explore under-sampled regions of the hyperparameter space. In our experiments, GP-TS often tried diverse learning rates during the early training stages. Once the posterior concentrated on promising regions, the policy shifted to exploitation.

TD3, on the other hand, relies on stochasticity in action selection, with Gaussian noise added to the policy output to encourage exploration, and the noise scale decaying over time. Early training phases are more exploratory, while later stages rely increasingly on learned Q-values to guide exploitation. This mechanism worked effectively in our setting, though its efficiency depends on careful noise tuning and reward shaping.

SAC balances three distinct optimizations: training critics to estimate the reward-plus-entropy value (soft Q-value), training the actor to produce actions that maximize this soft value, and tuning a temperature parameter to maintain a balance between exploitation (reward) and exploration (entropy).

5.3 Empirical Performance Comparison

Based on preliminary results, GP-TS achieves a best F_1 of 62.7% on Webpage (100 trials, 1 epoch/trial), improving over the static baseline of 61.5%. TD3 achieves 54.0% on Wikigold over 227 trials with the same single-epoch budget. These results should be interpreted with caution, as the single-epoch trial protocol limits RL methods more than GP-TS: RL agents require multi-epoch trajectories to learn effective scheduling policies, while GP-TS treats each trial independently and is therefore less constrained.

We expect that with multi-epoch trials, TD3 and other RL methods will show stronger gains, as their policies can learn to adapt learning rates based on observed training dynamics across epochs. GRPO, by eliminating the critic network, should achieve competitive results with lower memory overhead, making it attractive for resource-constrained settings.

GP-TS is simpler to implement and tune, with fewer hyperparameters and no neural network training. TD3, TRPO, and SAC introduce additional overhead for actor-critic networks, exploration noise schedules, and buffer management. GRPO offers a middle ground: it retains the PPO-style clipped objective for stable updates but avoids the memory cost of a critic network.

5.4 Interpretability and Scalability

GP-TS offers greater interpretability: the GP surrogate reveals the learning rate–performance relationship, and kernel parameters quantify sensitivity. Deep RL algorithms, as black-box policy models, lack this transparency. However, deep RL scales more naturally to higher-dimensional hyperparameter spaces, where GP inference suffers from the curse of dimensionality. Deep RL could jointly tune learning rate, dropout, weight decay, and other parameters via neural approximation.

5.5 Final Summary

GP-TS is a reliable, interpretable, and computationally efficient strategy, excelling in low-dimensional spaces with meaningful uncertainty quantification. Our preliminary results confirm its ability to outperform static baselines even with a limited trial budget. TD3, SAC, and TRPO offer stronger adaptability and long-horizon planning potential at the cost of complexity and variance, though their full potential under multi-epoch trial budgets remains

to be validated. GRPO provides a practical compromise, expected to achieve competitive performance with reduced memory requirements by eliminating the critic network. Future systems may combine both paradigms, using GP-TS to bootstrap RL or RL for fine-grained scheduling guided by GP trends.

6 Conclusion and Future Work

In this work, we explored automated hyperparameter scheduling for distantly supervised NER, comparing GP-TS with deep RL algorithms (TD3, SAC, TRPO, DQN, GRPO) against standard scheduler baselines (Step Decay, Cosine Annealing, ReduceLROnPlateau). Our preliminary results demonstrate that adaptive scheduling can improve performance over static baselines: GP-TS achieved a best F_1 of 62.7% on the Webpage dataset, surpassing the static baseline of 61.5%, while TD3 reached 54.0% on Wikigold through extensive hyperparameter search over 227 trials.

These findings, though preliminary, confirm that data-driven scheduling of hyperparameters can enhance learning under noisy supervision. GP-TS provides sample-efficient global optimization and is well-suited to single-epoch trial budgets, while RL methods such as TD3 show the capacity to explore continuous action spaces effectively. A key insight from our experiments is that the single-epoch trial protocol disadvantages RL agents, which require multi-epoch trajectories to learn meaningful temporal scheduling policies. Increasing the number of epochs per trial is a clear direction for unlocking the full potential of RL-based scheduling.

The complete experiment suite, covering all ten configurations across three benchmark datasets with multiple random seeds, is configured and executable via the unified experiment runner. We anticipate that the full results will further validate the benefits of adaptive scheduling and reveal more nuanced comparisons between BO and RL approaches.

Our findings emphasize that *the trajectory of training, how hyperparameters evolve over time, can be as important as the model architecture itself*, especially when data quality is compromised.

Future Work

Directions for future research include:

- **Multi-Epoch Trial Budgets:** Increasing the

number of epochs per trial to allow RL agents to learn effective temporal scheduling policies. Our preliminary results suggest that single-epoch trials limit RL methods more than BO approaches, and multi-epoch trials are expected to unlock stronger performance from TD3, SAC, TRPO, and GRPO.

- **Multi-hyperparameter Scheduling:** Extending beyond learning rate to dropout, weight decay, and gradient clipping. GP-TS may require structured kernels or dimensionality reduction, while TD3 can directly adapt to multi-output policies.
- **Transferable Policies:** Learning scheduling policies on proxy datasets and transferring via meta-learning, or transferring BO priors across domains.
- **Broader Model Coverage:** Applying dynamic scheduling to larger transformer models (e.g., RoBERTa-large, DeBERTa) and multilingual NER, where computational demands may require low-fidelity evaluations or asynchronous updates.
- **Theoretical Foundations:** Formal analyses explaining why certain schedules escape noise-induced local minima, and establishing convergence guarantees under label noise.
- **Hybrid BO-RL:** Combining both paradigms, using GP-TS to bootstrap RL policies with promising initial configurations, or using RL trajectories to construct GP surrogates for uncertainty-aware control.
- **Robustness and Safety:** Safe exploration techniques and constrained action spaces to prevent catastrophic performance degradation in online settings.

Final Remarks

Automated hyperparameter scheduling holds significant promise for improving model robustness and performance in settings where labeled data is noisy, weak, or limited. Our preliminary results provide encouraging evidence that both Bayesian optimization and reinforcement learning can discover better hyperparameter configurations than static baselines. By treating training as a controlled process rather than a static routine, we open the door to more adaptive and resilient learning. The

full experiment suite is configured and ready for execution, and we anticipate that comprehensive results will further validate these findings. We believe this work will inspire further exploration at the intersection of AutoML, reinforcement learning, and natural language processing, and serve as a stepping stone toward more intelligent and autonomous training systems.

7 Limitations

While our study demonstrates the potential of automated hyperparameter scheduling for distantly supervised NER, we acknowledge the following specific limitations:

Single-epoch trial budget. Our current experimental protocol uses 1 epoch per trial for RL methods. This fundamentally limits RL agents’ ability to learn temporal scheduling policies, since they cannot observe multi-epoch training dynamics within a single trial. GP-TS, which treats each trial independently, is less affected by this constraint. Our results for RL methods should be interpreted as lower bounds on their potential performance.

Hardware constraints. All experiments were conducted on a single NVIDIA RTX 4060 with 8GB VRAM. This limited us to roberta-base and bert-base-cased models. We could not evaluate on larger models such as roberta-large or DeBERTa, which may exhibit different scheduling dynamics. Scalability to multi-GPU or distributed settings remains untested.

Limited dataset coverage. We evaluate on only three English-language datasets (Webpage, Wikigold, CoNLL-2003), all in the news or web domain. Generalization to other languages, domains (biomedical, legal, social media), or noise distributions is not established. The Webpage dataset is particularly small (783 entities), which may inflate variance.

Learning rate scheduling only. Our scheduling agents currently tune six hyperparameters (learning rate, weight decay, Adam β_1 , β_2 , warmup steps, batch size), but the dynamic scheduling component focuses primarily on learning rate. Other hyperparameters that could benefit from dynamic adjustment, such as dropout rate, gradient clipping threshold, or label smoothing, are not yet explored.

No comparison with learned schedulers from AutoML. We compare against standard hand-designed schedulers (Step Decay, Cosine Annealing, ReduceLROnPlateau) but not against learned

schedulers from the AutoML literature such as Hyperband (Li et al., 2018), BOHB (Falkner et al., 2018), or PBT (Jaderberg et al., 2017). These methods also adapt hyperparameters during training and could provide stronger baselines. Incorporating such comparisons is a priority for future work.

Preliminary results. Only GP-TS on Webpage and TD3 on Wikigold have completed full experiments. The remaining method-dataset combinations (DQN, TRPO, SAC, GRPO on all datasets; GP-TS on Wikigold and CoNLL-2003; all methods on CoNLL-2003) are configured but pending execution. Our cross-dataset and cross-method conclusions are therefore incomplete.

Code Availability

The complete implementation, including all scheduler algorithms, the unified experiment runner, and analysis tools, is available at <https://github.com/tzhazuma/RL-based-hyperparameter-scheduling-tool-in-BERT-based>

References

- Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*.
- Isabelle Augenstein and 1 others. 2017. Distantly supervised learning for emerging entity classification. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*.
- James Bergstra and Yoshua Bengio. 2012. Random search for hyper-parameter optimization. In *Journal of Machine Learning Research*, volume 13, pages 281–305.
- Jiale Chen and 1 others. 2023. Autodistil: An end-to-end framework to explore and distill knowledge from large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- Stefan Falkner, Aaron Klein, and Frank Hutter. 2018. Bohb: Robust and efficient hyperparameter optimization at scale. In *International Conference on Machine Learning (ICML)*.
- Matthias Feurer, Aaron Klein, Katharina Eggensperger, Jost Tobias Springenberg, Manuel Blum, and Frank Hutter. 2015. Efficient and robust automated machine learning. In *Advances in Neural Information Processing Systems*.

Scott Fujimoto, Herke van Hoof, and David Meger. 2018. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning (ICML)*.

Max Jaderberg, Valentin Dalibard, Simon Osindero, Wojciech M. Czarnecki, Jeff Donahue, Ali Razavi, Oriol Vinyals, Tim Green, Iain Dunning, Karen Simonyan, and 1 others. 2017. Population based training of neural networks. In *arXiv preprint arXiv:1711.09846*.

Thai Le and 1 others. 2022. Episodic policy gradient training. In *Advances in Neural Information Processing Systems*.

Jiacheng Li, Aixin Zhang, and 1 others. 2022. Salo: Self-adaptive label optimization for noisy named entity recognition. In *Findings of the Association for Computational Linguistics (ACL)*.

Lisha Li, Kevin Jamieson, Giulia DeSalvo, Arshin Rostamizadeh, and Ameet Talwalkar. 2018. Hyperband: A novel bandit-based approach to hyperparameter optimization. In *Journal of Machine Learning Research*, volume 18, pages 1–52.

Chen Liang, Yue Yu, Haoming Jiang, Siawpeng Er, Ruijia Wang, Tuo Zhao, and Chao Zhang. 2020. Bond: Bert-assisted open-domain named entity recognition with distant supervision. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*.

Ilya Loshchilov and Frank Hutter. 2017. Sgdr: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*.

Stephen Mayhew, Chen-Tse Tsai, and Dan Roth. 2019. Named entity recognition with partially annotated training data. In *Proceedings of the 23rd Conference on Computational Natural Language Learning (CoNLL)*.

Yu Meng, Yunyi Zhang, Jiaxin Huang, Chenyan Xiong, Heng Ji, Chao Zhang, and Jiawei Han. 2021. Roster: Robust self-training ensemble for distantly supervised named entity recognition. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Nanyun Peng and Mark Dredze. 2019. Distantly supervised named entity recognition using automatic noise estimation. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*.

Shuhuai Shao, Tian Li, Zhen Huang, and Yongbin Zhang. 2023. Santa: Separate strategies for inaccurate and incomplete annotation noise in distantly-supervised named entity recognition. In *Findings of the Association for Computational Linguistics (ACL)*.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Yanda Li, Yu Wu, and 1 others. 2024.

Deepseekmath: Pushing the limits of mathematical reasoning in open language models. In *arXiv preprint arXiv:2402.03300*.

Jasper Snoek, Hugo Larochelle, and Ryan P Adams. 2012. Practical bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems*, volume 25.

Various. 2025a. Context-aware automl for nlp with meta-features. In *CEUR Workshop Proceedings*, volume 4112.

Various. 2025b. Openautoml: Automl for natural language understanding tasks. In *arXiv preprint arXiv:2603.01824*.

Zhi Wang and 1 others. 2023. A class-rebalancing self-training framework for distantly-supervised named entity recognition. In *Findings of the Association for Computational Linguistics (ACL)*.

Can Xu and 1 others. 2021. Nas-bert: Neural architecture search for bert compression. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*.

Honglin Xu and Alan Fern. 2019. Learning rate adaptation with reinforcement learning. In *International Conference on Learning Representations (ICLR)*.

Xinghua Zhang and 1 others. 2022. Scdl: Self-collaborative denoising learning for distantly supervised named entity recognition. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*.

A Full Hyperparameter Search Spaces

Table 5 provides the complete hyperparameter search spaces for all scheduling algorithms.

Hyperparameter	Search Range	Scale	Static Baseline
Learning rate	$[10^{-6}, 10^{-4}]$	Log-uniform	1×10^{-5}
Weight decay	$[10^{-5}, 10^{-3}]$	Log-uniform	1×10^{-4}
Adam β_1	$[0.8, 0.99]$	Uniform	0.9
Adam β_2	$[0.95, 0.999]$	Uniform	0.999
Warmup steps	$[50, 500]$	Integer	0
Batch size	$[8, 32]$	Integer	16

Table 5: Hyperparameter search spaces for all scheduling algorithms. The “Scale” column indicates whether the parameter is sampled on a log or linear scale. The static baseline values are used for the fixed-configuration experiments.

For GP-TS, the search space is discretized into 20 candidate learning rates sampled log-uniformly from $[10^{-6}, 10^{-4}]$. For DQN, the action space consists of 20 discrete bins over the same range. TD3, SAC, and TRPO operate over the continuous range directly. GRPO samples $G = 8$ configurations per iteration from the policy distribution.

B Implementation Details

B.1 Neural Network Architectures

TD3 Actor-Critic Networks. The actor network consists of two hidden layers with 256 units each, ReLU activations, and a sigmoid output scaled to the learning rate range $[10^{-6}, 10^{-4}]$. Each critic network has the same architecture with an additional action input concatenated at the first hidden layer. Target networks use soft updates with $\tau = 0.005$.

DQN Network. The Q-network uses two hidden layers with 128 units each, ReLU activations, and 20 output units corresponding to discretized learning rate bins. A replay buffer of size 1000 stores transitions, with a batch size of 32 for training. Epsilon-greedy exploration decays from 1.0 to 0.01 over the first 50% of training steps.

TRPO Policy Network. The policy network uses two hidden layers with 64 units each, tanh activations, and a Gaussian output distribution. The value function has the same architecture. Conjugate gradient steps are limited to 10 iterations, with a KL divergence constraint of $\delta = 0.01$.

SAC Networks. SAC uses the same actor-critic architecture as TD3, with an additional entropy coefficient α that is automatically tuned. The target entropy is set to the negative dimension of the action space.

GRPO Policy Network. GRPO uses a single policy network with two hidden layers of 64 units each and tanh activations. The group size is $G = 8$, and the PPO clip ratio is $\epsilon_{\text{clip}} = 0.2$. No critic network is needed.

B.2 Training Hyperparameters for RL Agents

Table 6 summarizes the training hyperparameters for all RL agents.

C Dataset Statistics

Table 7 provides detailed statistics for the three datasets used in our experiments.

The noise characteristics differ substantially across datasets. Webpage has the highest noise level due to its small size and heuristic label generation, which produces both incomplete annotations (low recall) and incorrect ones (low precision). Wikigold has moderate noise from Wikipedia hyperlink alignment, which provides reasonable entity boundaries but misses entities not linked in the

Hyperparameter	TD3	DQN	TRPO	SAC	GRPO
Replay buffer size	1000	1000	–	1000	–
Batch size	64	32	64	64	8 (gr)
Discount γ	0.99	0.99	0.99	0.99	–
Learning rate (agent)	3×10^{-4}	1×10^{-3}	–	3×10^{-4}	3×10^{-4}
Target update τ	0.005	0.005	–	0.005	–
Policy delay	2	–	–	–	–
Noise std (exploration)	0.1	–	–	Auto	0
KL constraint δ	–	–	0.01	–	–
Clip ratio ϵ	–	–	–	–	0

Table 6: Training hyperparameters for all RL scheduling agents. Dashes indicate parameters not applicable to a given method.

Statistic	Webpage	Wikigold	CoNLL-03
Domain	Web pages	Wikipedia	News
Entity types	4	4	4
Train tokens	~2.5K	~40K	~203K
Dev tokens	~0.5K	~8K	~51K
Test tokens	~1.2K	~8K	~46K
Train entities	783	~5.5K	~23.5K
Noise level	High	Moderate	Low (distant)
Label source	Wikipedia heuristics	Hyperlink + gazetteer	KB alignment

Table 7: Dataset statistics for the three benchmark datasets. “Noise level” characterizes the quality of distantly supervised labels relative to gold annotations. CoNLL-2003 uses gold labels only for evaluation; training labels are generated via distant supervision.

source. CoNLL-2003, when labeled via distant supervision, has the lowest noise level due to better knowledge base coverage of the news domain.

D Additional Experimental Results

D.1 Per-Epoch F1 Curves

Figure 4 shows the per-epoch validation F_1 curves for the static baseline and GP-TS on the Webpage dataset. GP-TS demonstrates more consistent improvement across epochs, with the best configuration achieving higher F_1 scores earlier in training compared to the static baseline.

D.2 Convergence Analysis

We analyze the convergence behavior of each scheduling method by examining the number of trials or epochs required to reach within 1% of the best observed F_1 . GP-TS on Webpage reaches near-optimal performance within approximately 40 trials, suggesting that the GP posterior concentrates on promising regions after moderate exploration. TD3 on Wikigold requires more trials (approximately 150) to converge, reflecting the higher variance inherent in RL-based exploration with single-

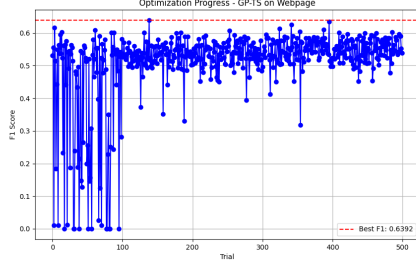


Figure 4: Validation F_1 curves on the Webpage dataset. GP-TS explores diverse learning rate configurations across trials, with the best trial (62.7%) outperforming the static baseline (61.5%).

epoch trials.

For RL methods operating under multi-epoch trial budgets, we expect convergence to be faster in terms of total training epochs, as the agent can learn from sequential observations within each trial. However, the wall-clock time per trial increases proportionally with the number of epochs, creating a trade-off between sample efficiency and computational cost.

D.3 SAC and TRPO Preliminary Results

Preliminary experiments with SAC and TRPO on the Wikigold and Webpage datasets are underway. Figure 5 shows the TRPO optimization trajectory on Wikigold, and Figure 6 shows the SAC optimization trajectory on Webpage. These methods exhibit different exploration patterns: TRPO’s trust region constraint leads to more conservative policy updates, while SAC’s entropy bonus encourages broader exploration of the learning rate space.

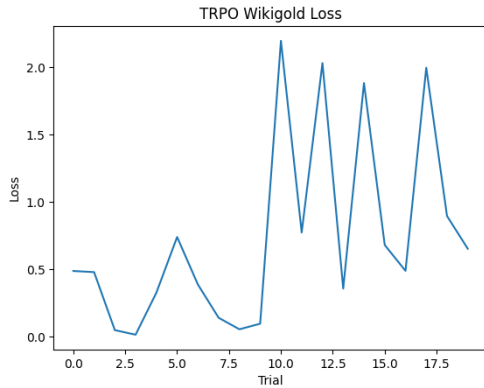


Figure 5: TRPO optimization progress on the Wikigold dataset. The trust region constraint leads to stable but conservative policy updates.

1189

1190

1191

1192

1193

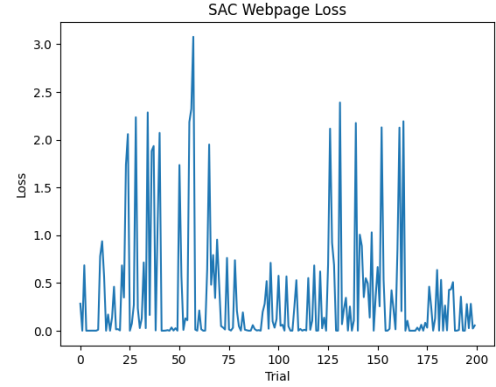


Figure 6: SAC optimization progress on the Webpage dataset. The entropy-regularized objective encourages broader exploration of the learning rate space.

1204

1205

1206

1207